

18th ACM International Conference on Web Search and Data Mining

Temporal Linear Item-Item Model for Sequential Recommendation

Seongmin Park, Mincheol Yoon, Minjin Choi, Jongwuk Lee

Sungkyunkwan University, Corea del Sur · DOI 10.1145/3701551.3703554

Miembros: **Cristóbal Moreno** · **Tomás Ketterer** · **Vicente Soto** · Sistemas Recomendadores (IIC3633)

Dos usuarios consumen **los mismos ítems en la misma secuencia**.
Uno lo hizo en una semana, el otro, en un año.
¿Deberíamos recomendarles **lo mismo**?



Figura 1: ejemplo de secuencia de usuario con distintos intervalos de tiempo (Beauty).

La idea que persigue el paper

La **secuencia** nos dice en qué orden pasaron las cosas, pero no *cuándo*. El **timestamp** muestra qué tan vigente es una preferencia: una compra de hace 2 días no pesa lo mismo que una de hace 6 meses.

El desafío es incorporar esa señal **sin el costo de un modelo neural**, manteniendo la eficiencia de un modelo lineal con solución cerrada.

¿Qué es Sequential Recommendation (SR)?

Predecir la **próxima interacción** del usuario a partir de su historial en orden cronológico, el modelo aprende **patrones de transición** entre ítems para capturar derivas de preferencia (*preference drifts*).

Next-item prediction



Con **feedback implícito** (clicks, compras, reproducciones, entre otros), el sistema produce un **ranking top-N** de candidatos para la posición siguiente.

Dos enfoques

- **Modelos neurales** (RNN, GNN, Transformers): Recorren la secuencia con **estados ocultos** (una memoria compacta de lo visto hasta ahora), la convierten en **grafo** o aplican **self-attention** (cada ítem pondera cuáles otros ítems de la secuencia importan más) para aprender dependencias complejas. *GRU4Rec* (Hidasi et al., 2015), *SR-GNN* (Wu et al., 2019), *SASRec* (Kang & McAuley, 2018).
- **Modelos lineales item-item** (*SLIST*): Aprenden una **matriz de transición/similitud entre ítems** mediante una regresión con **solución cerrada**, luego rankean candidatos combinando esos pesos item-item. *SLIST/SLIT* (Choi et al., 2021).

Pero ambos enfoques tienen un problema

Usan solo **información secuencial** (el orden: 1º, 2º, 3º...) y asumen implícitamente que **todos los intervalos de tiempo son iguales**.

En la práctica, el intervalo entre dos interacciones consecutivas varía de **minutos a meses**, y esa diferencia codifica cuán vigente sigue siendo el interés del usuario.

Información secuencial ≠ información temporal.

Los intervalos de tiempo **varían mucho**

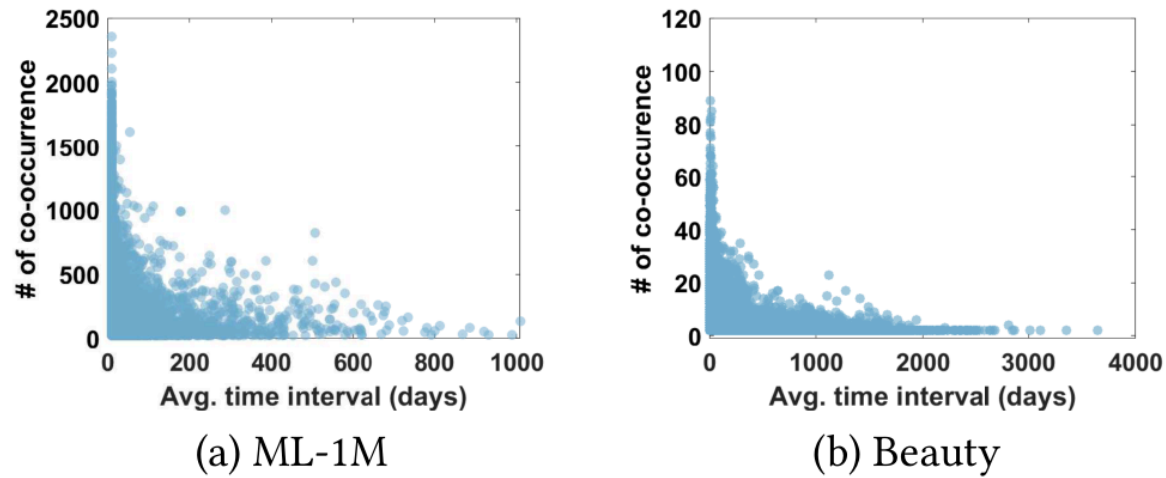


Figura 2: co-ocurrencias de ítems consecutivos según su intervalo promedio.

Desafío de eficiencia

Los SR neurales entrenan e infieren lento: el self-attention de un transformer es $O(L^2)$ en el largo L de la secuencia. Costoso para re-entrenar con frecuencia en producción.

Desafío de información temporal

Modelar solo el orden ignora que el mismo "gap posicional" puede representar **1 día o 188 días**. Sin timestamps no se capturan bien las derivas de preferencia.

Problema central

Los modelos eficientes suelen ignorar el tiempo real y los modelos que sí incorporan tiempo suelen ser más costosos.

TALE busca combinar ambas cosas: temporalidad y eficiencia.

Formulación del problema

Entradas del problema

Para cada usuario u de un conjunto $\mathcal{U}$ (m usuarios) sobre un catálogo $\mathcal{I}$ (n ítems):

$$\mathcal{S}^u = [i_1^u, i_2^u, \dots, i_{|\mathcal{S}^u|}^u] \quad \text{secuencia de ítems}$$

$$\mathcal{T}^u = [t_1^u, t_2^u, \dots, t_{|\mathcal{S}^u|}^u] \quad \text{timestamps reales}$$

Objetivo - Problema de optimización base

$$\Theta^* = \arg \max_{\Theta} \sum_{u=1}^m p(i_{|\mathcal{S}^u|+1}^u = \text{gt}(u) \mid \mathcal{S}^u, \mathcal{T}^u, \Theta)$$

Maximizar la probabilidad de que el ítem rankeado primero sea **el siguiente ítem real** del usuario.

Notación

$\mathcal{S}^u$ - historial ordenado del usuario u (feedback implícito).

$\mathcal{T}^u$ - timestamp de cada interacción (fecha y hora reales).

$\text{gt}(u)$ - *ground truth*: el ítem que el usuario efectivamente consumió después.

Θ - parámetros del modelo. En TALE será **una sola matriz** $B \in \mathbb{R}^{n \times n}$

m, n - número de usuarios e ítems.

Tiempo real sin perder eficiencia

La idea en una frase

TALE toma un **modelo lineal item-item**, le agrega **tiempo real** y **normalización temporal de popularidad**, pero **sin abandonar la solución cerrada**.

Cómo lo logra

- **Single-target augmentation:** elimina un sesgo posicional oculto del esquema multi-target.
- **Time interval-aware weighting:** usa el intervalo real entre eventos para ponderar la evidencia.
- **Trend-aware normalization:** corrige la popularidad usando una ventana temporal local.

Idea clave: estos tres cambios solo **re-pesan las matrices de entrenamiento**, por lo que la solución sigue siendo cerrada y pre-computable.

Qué cambia TALE

No rediseña la arquitectura desde cero: **reformula el entrenamiento** para que la señal temporal entre como pesos sobre las matrices, manteniendo el mismo problema.

Resultado: combina **eficiencia lineal**, **temporalidad** y **menos sesgo de popularidad** en un solo framework.

+18.71%

mejora máxima en
accuracy reportada

+30.45%

ganancia en ítems long-
tail

6.9x-

57x

más rápido que
neurales
eficientes

Tres familias de modelos previos

NEURALES SECUENCIALES

- **GRU4Rec** (Hidasi et al., 2015): lee la secuencia paso a paso con una **RNN**.
- **SASRec** (Kang & McAuley, 2018): usa *self-attention* para decidir qué parte del historial mirar.
- **FEARec** (Du et al., 2023) y **BSARec** (Shin et al., 2024): mejoran transformers para capturar patrones secuenciales más finos.

Limitación: solo modelan el orden, **no el timestamp**. Costo de transformer.

NEURALES TEMPORALES

- **TiSASRec** (Li, Wang & McAuley, 2020): mete el **tiempo entre eventos** dentro del attention.
- **TCPSRec** (Tian et al., 2022): busca aprender **patrones temporales repetidos**.
- **TiCoSeRec** (Dang et al., 2023): entrena con variaciones temporales para volver el modelo más robusto.

Limitación: ganan accuracy con tiempo, pero **no resuelven el conflicto de la eficiencia**.

LINEALES / EFICIENTES

- **SLIST/SLIT** (Choi et al., 2021): aprende relaciones **item-item** con solución cerrada.
- **LinRec** (Liu et al., 2023): abarata la atención para manejar secuencias largas.
- **LRURec** (Yue et al., 2024): reemplaza transformers por una recurrencia lineal más barata.

Limitación: mejoran escalabilidad o mantienen solución cerrada, pero **no incorporan el tiempo real como señal principal**.

Modelos lineales **item-item**

Toda la familia (EASE, SLIS, SLIT, TALE) se reduce a una **regresión ridge**: aprender una matriz B que reconstruye/predice interacciones.

$$\min_B \|Y - X B\|_F^2 + \lambda \|B\|_F^2$$

$$s_{u,j} = X_{u,*} \cdot \hat{B}_{*,j} \quad \hat{B} = (X^\top X + \lambda I)^{-1} X^\top Y$$

Intuición

- B_{jk} mide cuánta evidencia aporta el ítem j para recomendar el ítem k .
- El score de un candidato suma la contribución de todos los ítems del historial.
- El entrenamiento se obtiene con una sola resolución matricial, por solución cerrada.

Glosario

$X \in \{0,1\}^{m \times n}$ - matriz de entrada: $X_{u,j} = 1$ si u interactuó con j (multi-hot por fila).

Y - matriz objetivo: qué ítems debería predecir cada fila.

$B \in \mathbb{R}^{n \times n}$ - pesos ítem→ítem: el "modelo" completo.

λ - regularización L2 (ridge): evita sobreajuste y hace invertible la matriz.

$\|\cdot\|_F$ - norma de Frobenius: suma de cuadrados de todas las entradas.

SLIT: el modelo lineal secuencial

SLIST adapta la regresión para predecir **transiciones** de los ítems ya vistos (fuente S) hacia los que vienen (target T). TALE parte exactamente de aquí.

Multi-target augmentation

Cada secuencia $[i_1, i_2, i_3, i_4]$ se parte en pares:

$([i_1] \rightarrow [i_2, i_3, i_4])$

$([i_1, i_2] \rightarrow [i_3, i_4])$

$([i_1, i_2, i_3] \rightarrow [i_4])$

Position-aware weighting

Pondera por **cercanía posicional**: más peso a ítems cerca del final de la fuente y del inicio del target. Es decir, usa el **orden dentro de la secuencia** como proxy de recencia, pero **sin usar tiempo real**.

¿Por qué no basta?

- El peso por **posición** trata igual un gap de 1 día que uno de 188 días si ocupan la misma distancia en la secuencia.
- Multi-target **esconde un peso implícito** que premia pares lejanos, exactamente lo contrario de lo que la señal temporal necesita.

Motivación para TALE: incorporar **tiempo real** en lugar de estas heurísticas posicionales, sin perder la solución cerrada.

El framework de TALE

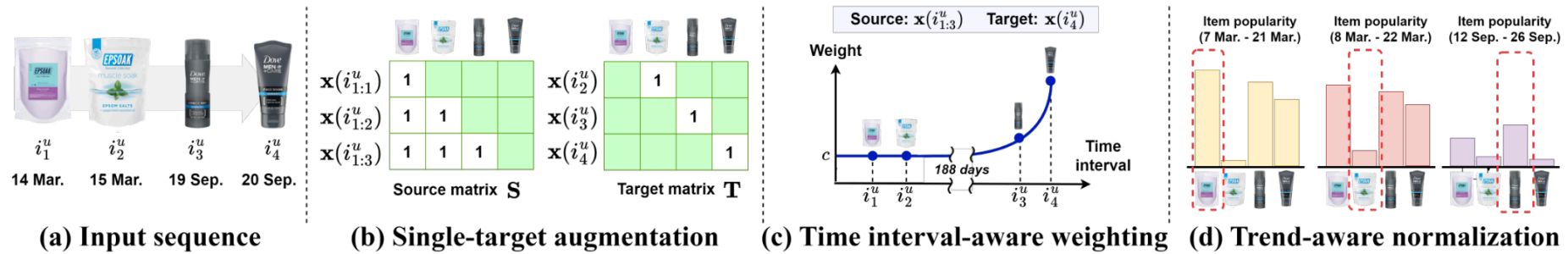


Figura 3: ilustración de las tres componentes de TALE.

$$\min_B \|\tilde{T} - \tilde{S} B\|_F^2 + \lambda \|B\|_F^2$$

donde $\tilde{T} = W_{\text{trend}} \odot T$, $\tilde{S} = W_{\text{time}} \odot W_{\text{trend}} \odot S$

$$\hat{B}_{\text{TALE}} = (\tilde{S}^\top \tilde{S} + \lambda I)^{-1} \tilde{S}^\top \tilde{T}$$

Qué cambia en el modelo

- TALE mantiene el **mismo objetivo ridge** del modelo lineal base, la diferencia es que entrena con $\tilde{S}$ y $\tilde{T}$, no con S y T sin modificar.
- Así, el tiempo entra al modelo **sin cambiar la forma de la solución cerrada**.

Glosario: $\tilde{S}$, $\tilde{T}$ son matrices re-pesadas, W_{time} y W_{trend} introducen tiempo real y popularidad local.

Single-target augmentation: elimina el sesgo posicional

El hallazgo teórico del paper

Expandiendo el objetivo de **multi-target** (SLIT), cada par fuente→target aparece con un peso implícito igual a su **distancia posicional** ($h - s$). Eso hace que **los pares lejanos pesen más**:

$$\hat{B}_{\text{multi}} = \arg \min_B \sum_{u=1}^m \sum_{s=1}^{|S^u|-1} \sum_{h=s+1}^{|S^u|} (h - s) \|x(i_h^u) - x(i_s^u)B\|_F^2 + \tilde{B}_S$$

Con target de **un solo ítem** (el siguiente), el término ($h - s$) desaparece. Así, **todos los pares parten iguales**:

$$\hat{B}_{\text{single}} = \arg \min_B \sum_{u=1}^m \sum_{s=1}^{|S^u|-1} \sum_{h=s+1}^{|S^u|} \|x(i_h^u) - x(i_s^u)B\|_F^2 + \tilde{B}_S$$

$x(i)$ one-hot del ítem i

s, h posiciones fuente/target

$\tilde{B}_S$ términos solo-fuente (iguales en ambos)

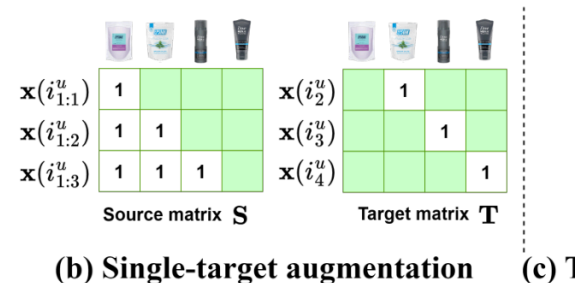


Figura 3(b): single-target augmentation.

Intuición

- La estrategia *multi-target* tiende a aumentar la importancia de correlaciones entre ítems distantes dentro de la secuencia.
- Eso puede dificultar que W_{time} modele de forma limpia el efecto del intervalo temporal.
- En contraste, *single-target* usa solo el siguiente ítem como objetivo.
- Así, aprende relaciones temporales más directas.

Time interval-aware weighting: **pondera por el intervalo real**

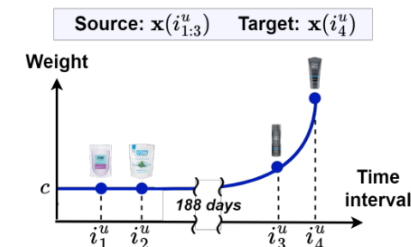
$$w_{\text{time}}(\mathcal{S}_k^u, i_s^u) = \max\left(\exp\left(-\frac{t_{|\mathcal{S}_k^u|}^u - t_s^u}{\tau_{\text{time}}}\right), c\right)$$

$t_{|\mathcal{S}_k^u|}^u - t_s^u$ - **intervalo real** (en tiempo) entre el ítem fuente i_s^u y el target de la sub-secuencia $\mathcal{S}_k^u$.

$\exp(-\Delta t/\tau)$ - **decaimiento exponencial**: mientras más reciente es una interacción, mayor es su peso; mientras más antigua, menor es su influencia.

τ_{time} - controla la **velocidad del olvido**: valores más grandes hacen que el peso caiga más lento; valores más chicos, más rápido.

$c \in [0, 1]$ - **piso del peso**: un ítem de hace 30 días no cae a $e^{-30/\tau} \approx 0$, conserva al menos c . Preserva la **dependencia de largo plazo** como señal colaborativa débil.



Time interval-aware weighting

Figura 3(c): time interval-aware weighting.

Intuición

- El modelo pregunta qué tan vigente sigue siendo cada interacción del historial.
- Las interacciones recientes pesan más; las antiguas pesan menos.
- El parámetro c evita que las interacciones lejanas desaparezcan por completo.
- Así, el tiempo real entra al modelo como una señal continua, no solo como orden posicional.

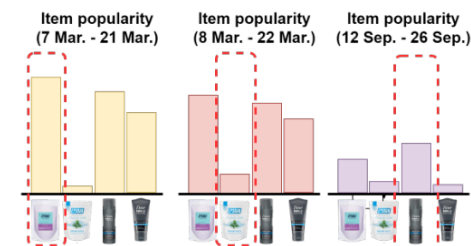
Trend-aware normalization: popularidad local en el tiempo

$$p_{u,j}(N) = \sum_{v=1}^m 1(|t_{u,j} - t_{v,j}| \leq N) \quad W_{\text{trend}} = (P_{\text{trend}})^{-\gamma}, \quad \gamma = \frac{1}{2}$$

$p_{u,j}(N)$ - cuántos usuarios interactuaron con el ítem j a $\pm N$ días de cuando lo hizo u .

$\gamma = 1/2$ - hace que los ítems **muy populares** queden más penalizados que los menos populares, pero sin borrar completamente su señal.

$N \rightarrow \infty$ - si la ventana temporal se hace infinita, deja de importar *cuándo* ocurrió la interacción y se recupera la **popularidad global acumulada**.



(d) Trend-aware normalization

Figura 3(d): trend-aware normalization.

Intuición

- TALE no considera solo si un ítem es popular, sino **cuándo** lo es.
- Un ítem de moda en un período corto no debería tratarse igual que uno popular de manera constante.
- Esta normalización se pre-computa, así que no agrega costo extra relevante.

Setup: datasets, métricas y baselines

Dataset	#Usuarios	#Ítems	#Interacc.	Densidad	Largo prom.	Intervalo prom.
ML-1M	6,040	3,953	999,611	5.19%	165.6	0.6 d
Beauty	22,363	12,101	198,502	0.07%	8.9	69.6 d
Toys	19,412	11,924	167,597	0.07%	8.6	86.0 d
Sports	29,858	18,357	296,337	0.06%	8.3	74.1 d
Yelp	30,494	20,061	317,078	0.05%	10.4	18.6 d

Leave-one-out

 $i_1 \dots i_{L-2}$ train i_{L-1} validación i_L test

Para probar el modelo, se le entrega el historial conocido del usuario y se le pide rankear **todo el catálogo**; luego se mide en qué posición quedó el ítem real de test.

Métricas

- **HR@K** (Hit Ratio): vale 1 si el ítem correcto aparece en el top-K, y 0 si no aparece.
- **NDCG@K**: también mira si el ítem correcto aparece en el top-K, pero da más crédito cuando queda **más arriba** en el ranking.
- **Head / Tail**: separa el test por popularidad en train - top 20% (head) vs bottom 80% (tail). Buen desempeño en tail sugiere **menos sesgo de popularidad**.

Los 10 baselines

NEURALES

SASRec · DuoRec · FEARec · BSARec

TEMPORALES

TiSASRec · TCPSRec · TiCoSeRec

EFICIENTES

LinRec · LRURec · SLIST

Hiperparámetros: se probaron varias combinaciones en validación y se eligió la que maximizó **NDCG@10**. En los modelos neurales, además, se detuvo el entrenamiento cuando la validación dejó de mejorar y se usó largo máximo de secuencia 50.

Resultados: gana en 4 de 5 datasets

Dataset	Mejor neural (NDCG@10)	SLIST	TALE	Imp. vs SLIST
ML-1M	0.1543 · BSARec	0.1377	<u>0.1500</u>	+8.9%
Beauty	0.0451 · FEARec	<u>0.0499</u>	0.0554	+11.0%
Toys	0.0500 · BSARec	<u>0.0599</u>	0.0661	+10.4%
Sports	0.0282 · TiCoSeRec	<u>0.0315</u>	0.0352	+11.8%
Yelp	0.0413 · BSARec	<u>0.0426</u>	0.0442	+3.8%

+9.07%

ganancia promedio NDCG@10 de TALE sobre
SLIST (5 datasets)

4 / 5

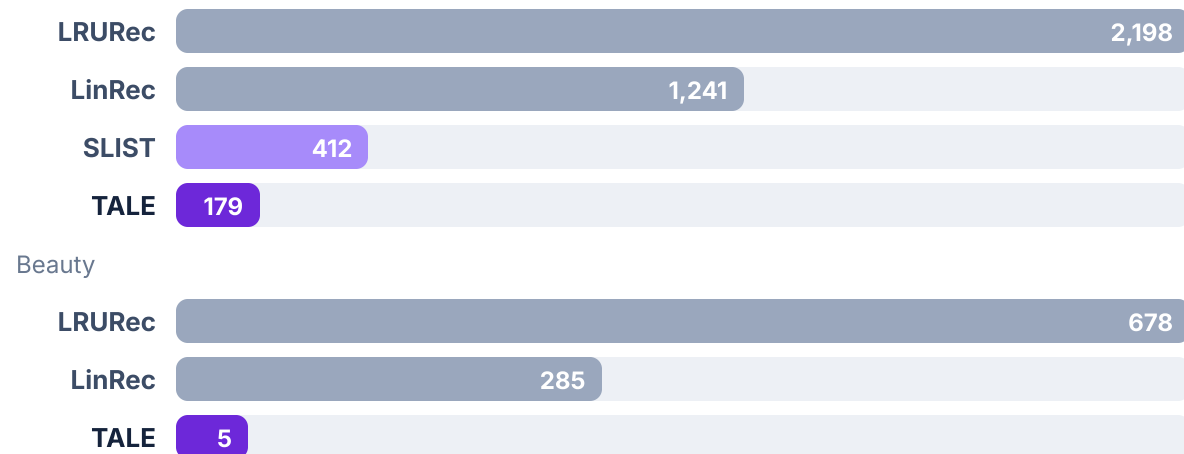
datasets donde un modelo lineal supera al
SOTA neural (BSARec)

Interpretaciones

- En los datasets más **dispersos** (Beauty, Toys, Sports, Yelp), TALE supera tanto a SLIST como al mejor baseline neural en NDCG@10.
- Agregar tiempo **sí ayuda**, pero no de cualquier forma: TALE supera a otros modelos temporales como TiSASRec y TCPSRec.
- En **ML-1M**, que es mucho más denso y tiene secuencias largas, los modelos neurales siguen siendo competitivos.

Eficiencia: entrenamiento e inferencia

Tiempo de entrenamiento (segundos)



6.9x – 57x

entrenamiento más rápido que LinRec (ML-1M · Beauty · Yelp: 6.9x / 57x / 17.7x)

145x

inferencia más rápida que LRURec en ML-1M (0.2 s vs 29 s)

Accuracy también mejora

HR@5	LinRec	LRURec	TALE
ML-1M	0.1843	0.1491	0.1854
Beauty	0.0550	0.0363	0.0674
Toys	0.0631	0.0624	0.0815
Sports	0.0314	0.0178	0.0418
Yelp	0.0414	0.0269	0.0558

Diferencia central

- **Transformers:** entrenan por gradiente durante muchas epochs y su costo crece con el largo de la secuencia.
- **TALE:** pre-computa los pesos temporales y luego resuelve **una sola regresión ridge**.
- En inferencia, TALE solo calcula un **score item-item** a partir del historial del usuario.

Análisis: popularidad y ablation

Sesgo de popularidad

NDCG@5	Tail	ML-1M		Beauty	
		Head	Tail	Head	Tail
BSARec	0.0740	0.1642	0.0169	0.0548	
SLIST	0.0624	0.1559	0.0235	0.0676	
SLIST+Norm.	0.0675	0.1290	0.0275	0.0463	
TALE	0.0814	<u>0.1612</u>	<u>0.0271</u>	0.0736	

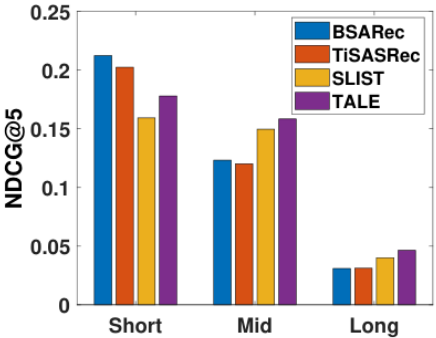
TALE mejora mucho en **ítems poco populares** y, al mismo tiempo, **no pierde** en los ítems populares. En cambio, la normalización clásica mejora tail, pero empeora head.

Ablation: ¿aportan los 3 componentes?

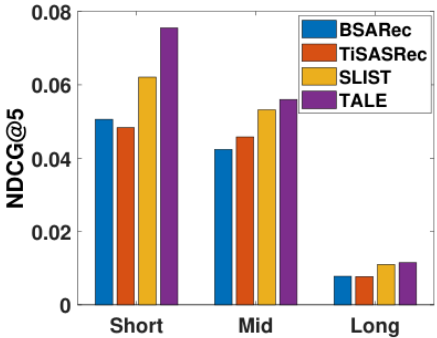
single-tgt	time-w	trend-n	ML-1M	Beauty
✓	✓	✓	0.1275	0.0485
✗	✓	✓	0.0473	0.0296
✓	✗	✓	0.1227	0.0450
✓	✓	✗	0.1257	0.0468
ninguno (= SLIT base)			0.1164	0.0435

Los tres componentes ayudan, pero uno es especialmente crítico, si se quita **single-target** y se deja el peso temporal, el rendimiento cae fuerte. Es decir, el peso temporal funciona bien **solo si primero se corrige el sesgo de multi-target**.

Intervalos de tiempo e hiperparámetros



(a) ML-1M



(b) Beauty

Figura 4: accuracy por grupo de intervalo temporal (Short/Mid/Long).

Óptimos por dataset

	λ	c	τ_{time}	N (días)
ML-1M	100	0.2	1/512	180
Beauty	100	0.4	1/2	180
Toys	100	0.3	1	360
Sports	500	0.4	4	180
Yelp	0.001	0.2	1/32	180

Interpretación

- Los mejores hiperparámetros dependen del dataset: algunos requieren olvidar rápido y otros conservar más memoria.
- Una **ventana temporal finita** funciona mejor que mezclar toda la historia acumulada.
- En la práctica, **c**, **τ** y **N** deben ajustarse por dataset.

Cuatro conclusiones

- 1 El orden por sí solo no es suficiente.** Dos ítems consecutivos pueden estar separados por 1 día o por 188, y esa diferencia temporal codifica información relevante sobre la vigencia de la preferencia del usuario.
- 2 TALE incorpora temporalidad sin abandonar el marco lineal.** Para ello, re-pesa las matrices de una regresión ridge mediante W_{time} , W_{trend} y target único, manteniendo una solución cerrada pre-computable.
- 3 El detalle teórico es importante.** El esquema multi-target introduce un peso posicional implícito $(h - s)$ que interfiere con la señal temporal, y el estudio de ablation confirma que corregir ese sesgo es fundamental para el desempeño del modelo.
- 4 Los resultados muestran una mejora consistente en precisión, eficiencia y sesgo de popularidad.** En promedio, TALE supera a SLIST en alrededor de 9%, alcanza mejoras de hasta 30% en long-tail y entrena hasta 57 veces más rápido que modelos neurales eficientes.

Referencias

PAPER PRESENTADO

Park, S., Yoon, M., Choi, M., & Lee, J. (2025). Temporal Linear Item-Item Model for Sequential Recommendation. *WSDM '25*. DOI 10.1145/3701551.3703554 · arXiv:2412.07382

BASE LINEAL

[5] Choi et al. (2021). Session-aware Linear Item-Item Models for Session-based Recommendation. *WWW*.

[27] Steck (2019). Embarrassingly Shallow Autoencoders for Sparse Data. *WWW*.

[11] Garg et al. (2019). STAN: Sequence and Time Aware Neighborhood. *SIGIR*.

SR TEMPORAL

[19] Li, Wang & McAuley (2020). Time Interval Aware Self-Attention (TiSASRec). *WSDM*.

[30] Tian et al. (2022). Temporal Contrastive Pre-Training (TCPSRec). *CIKM*.

[7] Dang et al. (2023). Uniform Sequence Better (TiCoSeRec). *AAAI*.

SR NEURAL

[12] Hidasi et al. (2015). Session-based Recommendations with Recurrent Neural Networks (GRU4Rec). *ICLR Workshop / arXiv*.

[17] Kang & McAuley (2018). Self-Attentive Sequential Recommendation (SASRec). *ICDM*.

[38] Wu et al. (2018). Session-based Recommendation with Graph Neural Networks (SR-GNN). *AAAI*.

[23] Qiu et al. (2021). Contrastive Learning for Representation Degeneration (DuoRec). *WSDM*.

[25] Shin et al. (2023). An Attentive Inductive Bias for Sequential Recommendation beyond the Self-Attention (BSARec). *AAAI*.

[8] Du et al. (2023). Frequency Enhanced Hybrid Attention (FEARec). *SIGIR*.

EFICIENCIA Y DEBIASING

[20] Liu et al. (2024). LinRec: Linear Attention Mechanism for Long-term SR. *SIGIR*.

[39] Yue et al. (2023). Linear Recurrent Units for SR (LRURec). *WSDM*.

[3] Chen et al. (2023). Bias and Debias in Recommender System: A Survey. *TOIS*.

[13] He et al. (2020). LightGCN. *SIGIR*.

Glosario

$\mathcal{S}^u$ - secuencia de ítems del usuario u .

$\mathcal{T}^u$ - timestamps reales asociados a la secuencia del usuario u .

B - matriz item-item del modelo, cada entrada representa cuánta evidencia aporta un ítem para recomendar otro.

$\tilde{S}, \tilde{T}$ - matrices de fuente y target re-pesadas por tiempo y popularidad.

$W_{\text{time}}, W_{\text{trend}}$ - pesos que introducen intervalo temporal real y popularidad local.

λ - regularización ridge que estabiliza el aprendizaje de la matriz B .

$x(i)$ - vector one-hot del ítem i .

τ_{time} - controla qué tan rápido decae el peso de una interacción antigua.

c - piso mínimo del peso temporal, evita que interacciones lejanas desaparezcan por completo.

$p_{u,j}(N)$ - popularidad del ítem j alrededor del momento en que interactuó el usuario u .

N - tamaño de la ventana temporal usada para medir popularidad local.

γ - intensidad con que se penaliza la popularidad en la normalización temporal.

HR@K / NDCG@K - **HR@K** mide si el ítem correcto aparece en el top-K. **NDCG@K** además recompensa que aparezca más arriba en el ranking.

Head / Tail - partición del test entre ítems populares y poco populares, usada para analizar sesgo de popularidad.

Uso de **inteligencia artificial**

Para preparar esta presentación nos apoyamos en herramientas de IA generativa como asistente de trabajo. Específicamente en la preparación de la presentación a nivel de diseño

En qué nos apoyó la IA

- Extraer las figuras del paper y ordenar el contenido.
- Generar una primera versión del diseño y la maqueta de las diapositivas.
- Apoyar la redacción y proponer formas de explicar las fórmulas.

Qué decidimos nosotros

- La lectura y comprensión del paper, y qué incluir en cada slide.
- La estructura, el hilo conductor y el énfasis técnico de la exposición.
- La revisión de cada ecuación, tabla y resultado contra el paper original.
- Todas las correcciones y la versión final de la presentación.